



4 Proposed System

Figure 4.1: Overall Approach

This section describes the end-to-end journey of a badminton clip through the *current* IPD system (January 2026 codebase). The system is designed around a dual-pipeline architecture:

- **Pose Pipeline (Pose-LSTM)**: lightweight, uses only normalized pose (99D)
- **Hybrid Pipeline (Hybrid-TCN)**: richer, uses normalized pose (99D) + CNN ROI features (64D) → 163D fusion

Both pipelines share the same preprocessing philosophy: contact-centered segmentation, robust pose extraction, and sliding-window sequence modeling.

In the current implementation, the *classification* stage and the *biomechanical scoring* stage are intentionally decoupled:

- Classification is learned (deep sequence models).
- Scoring uses a research-grade analytic engine (KSI) that aligns expert vs user motion in time using DTW and then scores pose + dynamics.

Step 1: Video Ingestion + Clip Segmentation

Raw input is a video clip (file or webcam). Instead of processing the entire clip, the system processes a **fixed-duration segment** selected by config-driven rules (to focus on the biomechanically relevant portion of the shot).

Method in Detail (Implemented):

- The segment start and duration are computed using the rules in `params.yaml`:
 - Default: **tail window** (last ~1.75s)
 - Shot-specific overrides:
 - `forehand_clear` : tail window of ~2.0s
 - `forehand_lift` : middle window (centered) of ~2.0s

Why it matters:

- Badminton shots contain pre/post motion that can add noise.
- Fixed-length segmentation enables consistent sequence modeling without padding/truncation complexity.

Mathematical Formulation (Segment Selection)

Let a video have FPS f and N total frames. Given a configured segment length in seconds s , the target segment length in frames is:

$$M = \lfloor s \cdot f \rfloor$$

Tail-window extraction uses:

$$t_0 = \max(0, N - M), \quad t \in [t_0, t_0 + M)$$

Middle-window extraction uses:

$$t_0 = \max\left(0, \left\lfloor \frac{N}{2} \right\rfloor - \left\lfloor \frac{M}{2} \right\rfloor\right), \quad t \in [t_0, t_0 + M)$$

Step 2: Spatial Cropping + Robust Pose Tracking

The current system does **not** rely on “top-half-of-screen player selection” as the primary method. Instead it uses:

1. **Config-driven frame cropping** to remove irrelevant borders and stabilize the pose detector.
2. **MediaPipe Pose tracking** to extract landmarks per frame.

Method in Detail (Implemented):

- A base crop config (`top` , `bottom` , `left` , `right`) is applied to reduce background and camera framing variance.
- A per-video override mechanism adjusts crop parameters (especially `bottom`) based on shot type and video index.
- Cropping can be skipped automatically for filenames matching `"name (N).mp4"` patterns.

Pose extraction outputs:

- For training preprocessing:
 - Pose pipeline uses **pose_world_landmarks** (world-space 3D) and then normalizes.
 - Hybrid pipeline uses **pose_landmarks** (image-space normalized coords) and then normalizes.
- For inference + KSI evaluation, the system uses **image-space landmarks** for compatibility with expert templates.

Robustness behavior (Implemented):

- If pose is missing in a frame, the system reuses the **last valid pose** (otherwise zeros).

Mathematical Formulation (Crop + ROI)

Base cropping is a deterministic mapping from original image coordinates (x, y) to cropped image coordinates, parameterized by fractional margins (τ, β, ℓ, r) (top/bottom/left/right):

$$y \in [\tau H, H - \beta H), \quad x \in [\ell W, W - r W)$$

For the Hybrid pipeline ROI, let $\mathcal{J}$ be a set of joint indices (default shoulders→ankles) and let v_j be the landmark visibility for joint j .

Visible joints are:

$$\mathcal{J}_v = \{j \in \mathcal{J} : v_j \geq v_{\min}\}$$

If $|\mathcal{J}_v| < J_{\min}$ the system falls back to the previous ROI (if allowed) or the full cropped frame.

Otherwise, with landmark pixel coordinates (x_j, y_j) :

$$x_1 = \min_{j \in \mathcal{J}_v} x_j, \quad x_2 = \max_{j \in \mathcal{J}_v} x_j, \quad y_1 = \min_{j \in \mathcal{J}_v} y_j, \quad y_2 = \max_{j \in \mathcal{J}_v} y_j$$

The box is expanded by a margin m and then size-clamped to at least a fraction ρ of the frame size:

$$[x_1, x_2] \leftarrow [x_1 - m\Delta x, x_2 + m\Delta x], \quad [y_1, y_2] \leftarrow [y_1 - m\Delta y, y_2 + m\Delta y]$$

Temporal smoothing is applied via an exponential moving average toward the previous box B_{t-1} :

$$B_t \leftarrow (1 - \lambda)B_t + \lambda B_{t-1}$$

where λ is the configured smoothing factor.

Step 3: Feature Engineering + Sliding Window Formation

The system converts frame-level signals into fixed-length sequences for temporal learning.

3.1 Geometric Pose Normalization (99D)

Each frame's 33 keypoints (x,y,z) is normalized into a person-centric coordinate system:

- **Centering:** hip-center translation
- **Alignment:** rotate so the torso defines the vertical axis
- **Scaling:** normalize by spine length

This yields **99 features per frame** (33×3).

Mathematical Formulation (Geometric Normalization)

Let the raw landmark matrix for a frame be $P \in \mathbb{R}^{33 \times 3}$. Define hip center:

$$c = \frac{P_{LH} + P_{RH}}{2}$$

Center the pose: $\tilde{P} = P - c$.

Define a spine direction using the shoulder center:

$$s = \frac{\tilde{P}_{LS} + \tilde{P}_{RS}}{2}, \quad \ell = \|s\|_2$$

If ℓ is too small, the centered pose is used as-is. Otherwise, build an orthonormal basis $(\hat{x}, \hat{y}, \hat{z})$ where:

$$\hat{y} = \frac{s}{\ell}$$

and $\hat{x}$ is formed from the shoulder axis with its projection on $\hat{y}$ removed, then normalized (with a safe fallback if degenerate). Finally:

$$R = [\hat{x}; \hat{y}; \hat{z}] \in \mathbb{R}^{3 \times 3}, \quad P' = \tilde{P} R^T, \quad \bar{P} = \frac{P'}{\ell}$$

The normalized per-frame feature vector is $\text{vec}(\bar{P}) \in \mathbb{R}^{99}$.

3.2 Hybrid ROI + CNN Features (64D)

In the Hybrid pipeline, the system additionally extracts a compact visual embedding:

- A **pose-guided, forgiving ROI** is computed from full-body joints (shoulders $\rightarrow$ ankles) with:
 - visibility filtering
 - minimum-joint requirement
 - margin expansion
 - minimum box size
 - temporal smoothing (EMA)
 - fallback to last ROI or full frame if pose is unreliable
- The cropped ROI is resized and passed through **MobileNetV2** (ImageNet weights) and projected to **64D**, then L2-normalized.

Final per-frame hybrid feature vector:

- **Hybrid feature dim** = 99 (pose) + 64 (CNN) = **163D**

Mathematical Formulation (CNN Embedding)

Let I_t be the cropped (or ROI-cropped) RGB frame at time t , resized to 224×224 . The embedding is:

$$h_t = \text{MobileNetV2}(I_t) \in \mathbb{R}^d$$

$$u_t = \text{ReLU}(W h_t + b) \in \mathbb{R}^{64}$$

$$c_t = \frac{u_t}{\|u_t\|_2}$$

The fused hybrid feature is:

$$x_t = [\text{vec}(\bar{P}_t) \parallel c_t] \in \mathbb{R}^{163}$$

3.3 Sliding Window Sequences

Both pipelines use temporal windows:

- **Sequence length:** 40 frames
- **Stride:** 5 frames

Windows are saved incrementally during preprocessing (streaming) for efficiency.

Mathematical Formulation (Windowing)

Given a per-frame feature sequence $x_0, \dots, x_{T-1}$, window length L and stride S , the k -th window is:

$$X^{(k)} = [x_{kS}, x_{kS+1}, \dots, x_{kS+L-1}] \in \mathbb{R}^{L \times D}$$

Windows are produced while $kS + L \leq T$.

Hybrid preprocessing artifacts (implemented): each saved `.npz` window contains:

- `features` : $X^{(k)}$ (either $D = 99$ pose-only or $D = 163$ hybrid)
 - `raw_landmarks` : $(L, 33, 3)$ (hybrid pipeline) used downstream for KSI
 - `fps` : sampling rate metadata
-

Step 4: Shot Classification + Biomechanical Feedback (KSI)

4.1 Shot Classification

The system performs multi-class shot classification (6 classes in the current dataset):

- `forehand_clear`
- `forehand_drive`
- `forehand_lift`
- `forehand_net_shot`
- `backhand_drive`

- backhand_net_shot

Models (Implemented):

- **Pose Pipeline:** Conv1D + LSTM network trained on pose-only sequences.
- **Hybrid Pipeline:** 2 Kernel TCN -> GRU + GRU trained on hybrid sequences.

Contact-aware classification (Implemented in evaluation):

Instead of averaging predictions across all windows, evaluation focuses on the window containing the **contact moment**, detected by peak arm acceleration.

- Joints used: right shoulder (12), right elbow (14), right wrist (16)
- Signal: composite velocity → acceleration peak
- The predicted class is taken from the window containing this peak.

Mathematical Formulation (Probabilities + Contact Window)

Given a window $X^{(k)}$, the classifier outputs logits $z \in \mathbb{R}^C$ for C shot classes. The predicted probabilities are:

$$p(c \mid X^{(k)}) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

and the predicted label is:

$$\hat{c}^{(k)} = \arg \max_c p(c \mid X^{(k)})$$

Contact selection (as implemented in `src/evaluate_video.py`): for each candidate window, compute a composite arm velocity from right shoulder/elbow/wrist:

$$v_t = 0.5 \|\Delta w_t\|_2 + 0.3 \|\Delta e_t\|_2 + 0.2 \|\Delta s_t\|_2$$

Apply EMA smoothing to v_t and score the window by the overall magnitude of velocity change (a proxy for acceleration):

$$A = \|\Delta v\|_2$$

The system selects the window index with maximum A and uses its prediction as the primary shot label.

4.2 Posture Analysis: KSI

After classification, the system compares the user's kinematics to an expert template for the predicted shot.

Expert reference:

- Expert templates are stored in `data/expert_templates.npz` and loaded during evaluation.

KSI (Implemented):

- Extracts **32 hierarchical biomechanical features** per frame (angles, ratios, spine alignment, rotational dynamics, wrist dynamics, COM proxies)
- Computes temporal derivatives (velocity/acceleration/jerk) for motion quality
- Performs **phase-aware analysis** (preparation → loading → acceleration → contact → follow-through)
- Produces:
 - overall KSI scores
 - per-phase scores
 - per-joint error trajectories
 - confidence intervals / reliability flags
 - targeted recommendations

Mathematical Formulation (KSI Pipeline)

KSI compares an expert landmark sequence $E \in \mathbb{R}^{T_e \times 33 \times 3}$ and a user landmark sequence $U \in \mathbb{R}^{T_u \times 33 \times 3}$.

1. Per-frame feature extraction (32D)

For each frame, a 32D biomechanical feature vector is computed:

$$f_t = \phi(\text{landmarks}_t) \in \mathbb{R}^{32}$$

where $\phi(\cdot)$ includes joint angles (via 3D cosine law), limb ratios, spine alignment, and rotational dynamics. Example angle at point b :

$$\angle abc = \arccos \left(\frac{(a - b) \cdot (c - b)}{\|a - b\|_2 \|c - b\|_2} \right)$$

The trunk rotation velocity proxy is computed over the sequence using a smoothed gradient.

2. Contact-centered windowing (inside KSI)

KSI performs its own contact-centered cropping around the detected contact phase. If the contact center is t_c , the KSI window is:

$$[t_c - P, t_c + Q)$$

with defaults $P = 18$ and $Q = 18$ frames.

3. DTW alignment (feature space)

Let $F^E \in \mathbb{R}^{n \times 32}$ and $F^U \in \mathbb{R}^{m \times 32}$ be the extracted feature sequences after contact-windowing. DTW computes an alignment path minimizing cumulative L2 costs with a Sakoe–Chiba band constraint.

Cost matrix recurrence (implemented in `dynamic_time_warping_optimized`):

$$D_{0,0} = 0, \quad D_{i,0} = D_{0,j} = \infty$$

$$D_{i,j} = \|F_i^E - F_j^U\|_2 + \min\{D_{i-1,j}, D_{i,j-1}, D_{i-1,j-1}\}$$

Backtracking yields a monotone path $\pi = \{(i_k, j_k)\}_{k=1}^K$ and aligned sequences:

$$\tilde{F}_k^E = F_{i_k}^E, \quad \tilde{F}_k^U = F_{j_k}^U$$

4. Phase-aware temporal attention

KSI assigns higher weight to acceleration/contact. For each phase segment $[a, b)$, a Gaussian-shaped weight is applied within the segment and scaled by a phase importance factor w_{phase} .

Weights are normalized to preserve scale:

$$\sum_{k=1}^K \alpha_k = K$$

5. Component similarities (pose / velocity / acceleration)

Pose similarity uses attention-weighted cosine similarity:

$$S_{\text{pose}} = \frac{\sum_k \alpha_k \cos(\tilde{F}_k^E, \tilde{F}_k^U)}{\sum_k \alpha_k}$$

Velocity similarity computes first differences and combines direction similarity with a Gaussian magnitude penalty:

$$V_k^E = \tilde{F}_k^E - \tilde{F}_{k-1}^E, \quad V_k^U = \tilde{F}_k^U - \tilde{F}_{k-1}^U$$

$$S_{\text{vel}} = \frac{\sum_k \alpha_k \cos(V_k^E, V_k^U) \exp(-0.1(\|V_k^E\|_2 - \|V_k^U\|_2)^2)}{\sum_k \alpha_k}$$

Acceleration similarity computes second differences and compares magnitudes:

$$A_k^E = V_k^E - V_{k-1}^E, \quad A_k^U = V_k^U - V_{k-1}^U$$

$$S_{\text{acc}} = \frac{\sum_k \alpha_k \exp(-0.1(\|A_k^E\|_2 - \|A_k^U\|_2)^2)}{\sum_k \alpha_k}$$

6. Final KSI score

With configured weights (w_p, w_v, w_a) from `params.yaml` :

$$\text{KSI} = w_p S_{\text{pose}} + w_v S_{\text{vel}} + w_a S_{\text{acc}}$$

The implementation also computes jerk-based smoothness and phase-wise scores, and estimates confidence via bootstrap resampling of frame indices.

4.3 Coaching Output (Optional Natural Language Coach)

If enabled, the system converts KSI outputs into human-readable coaching guidance using an internal biomechanics knowledge base:

- prioritized fixes
 - cause → explanation → drill suggestions
 - phase-specific feedback
-

Offline Training (How Models Are Trained)

The system is trained on the saved window tensors produced by preprocessing.

Dataset Construction

Let $\{(X_i, y_i)\}_{i=1}^N$ be window samples where $X_i \in \mathbb{R}^{L \times D}$ and $y_i \in \{1, \dots, C\}$.

- Pose pipeline loads `features` windows with $D = 99$.
- Hybrid pipeline loads `features` windows with $D = 163$ and splits them into pose vs CNN:

$$X^{\text{pose}} = X[:, :, 0 : 99], \quad X^{\text{cnn}} = X[:, :, 99 : 163]$$

Objective Function

Models are trained with categorical cross entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log p(c \mid X_i)$$

Optimization + Early Stopping

Training uses Adam (as compiled in the model builders) and monitors validation accuracy with:

- early stopping (restore best weights)
- checkpointing to `models/*.h5`
- experiment logging via MLflow + DVCLive (DVC experiments)
- Rigorous Hyperparameter tuning using optuna

Splits

Both training scripts implement stratified splits:

- 20% test
 - 10% validation (from the remaining 80%)
-

Evaluation (How Predictions + KSI Are Computed)

For a given user clip, the evaluation script performs:

1. **Window extraction:** generate sliding windows and also retain raw landmarks per window.
 2. **Quality filtering:** drop windows with low pose availability (requires $\geq 70\%$ frames with detected pose) or invalid values.
 3. **Model inference:** run the trained classifier on all remaining windows (with automatic input-shape adaptation).
 4. **Contact-based selection:** choose the most reliable window using the arm-acceleration criterion and report that prediction.
 5. **KSI computation:** compute KSI for each window against the corresponding expert template and summarize (the script averages KSI across windows).
 6. **Coaching report** (optional): transform KSI outputs into structured feedback.
-

Expert Template Generation (DTW-Aligned Landmark Templates)

Expert templates are generated from multiple expert videos per class:

1. Apply the same preprocessing (segment bounds + crop overrides).
2. Extract raw landmarks and 32D enhanced features.
3. Select a reference sequence (minimum total DTW distance to others).
4. DTW-align all sequences to the reference in feature space.
5. Apply the same DTW path to align landmark sequences.
6. Compute a (quality-weighted) average landmark template.

This produces templates stored as raw landmarks $(T, 33, 3)$ so KSI can re-extract features consistently.

System Outputs (What the user gets)

For a given clip, the current system can produce:

- Predicted shot class + probabilities
 - Contact moment estimate (window + frame)
 - KSI score + component breakdown
 - Per-joint / per-feature error localization
 - Phase-wise scoring (preparation/loading/acceleration/contact/follow-through)
 - A structured coaching report (text + JSON), when enabled
-